

exp-mlkem-f203-stage3-routea-mod-vec

F203 Stage3 RouteA + mod q 向量预研方案

ascendc 工程（预研）

2026-05-19

摘要

本文档为 exp-mlkem-f203-stage3-routea-mod-vec/ 的计划书闭环:mat_c [16,512] int32 (Stage2 Cube 输出) $\rightarrow$ RouteA $\rightarrow$ Stage3.1 mod $q \rightarrow$ out [8,256] int32。在查阅 CANN 9.0.0 Ascend C API 与 dav_c220 实现后, Atlas A2 上无法对 RouteA 合并与 Stage3.1 做到 100% 单向量而保持 int64/除法语义; 本用例采用向量 GM 搬运 + 标量 int64 合并与 mod 的混合实现, 并在下表明确标量缺口与备选 API 路径。CPU 对拍: bash run.sh -r cpu -v Ascend910B4; 1 / 2 / 8 个 Vector AI Core 三档 (见 §2.1)。

目录

1	数学语义	2
1.1	输入输出	2
1.2	RouteA (对齐 f203_stage3_route_a)	2
1.3	Stage3.1 (mod q , stage31_mod)	2
2	向量实现规划	2
2.1	AI Core 规模与验证档位	2
2.2	分块与 Gather 策略	3
3	全量 API 清单	3
4	逐步覆盖率评估	4
4.1	为何 int32 向量不能安全承担 raw	5
4.2	Stage3.1 向量备选 (未采用)	5
4.3	本版实现闭环结论	5
5	实现与验证	5

1 数学语义

1.1 输入输出

- 输入 `mat_c_gm`: $[16, 512]$ int32。行 0..7 为 HI 乘积行, 8..15 为 LO 乘积行 (与 F203 Stage2 一致)。
- 输出 `out_gm`: $[8, 256]$ int32, 系数 $\in [0, q)$, $q = 3329$ 。

1.2 RouteA (对齐 f203_stage3_route_a)

对每条 poly $p \in [0, 7]$ 、输出列 $j \in [0, 255]$, 令偶/奇列 $c_e = 2j$, $c_o = 2j + 1$:

$$hh = C[p, c_e], \quad hl = C[p + 8, c_e] \quad (1)$$

$$lh = C[p, c_o], \quad ll = C[p + 8, c_o] \quad (2)$$

$$raw = hh \cdot 4096 + (hl + lh) \cdot 64 + ll \quad (\text{int64}) \quad (3)$$

1.3 Stage3.1 (mod q , stage31_mod)

本工程采用的主公式 (CANN 9.0.0, 假定整除/取整语义正确):

$$t = \begin{cases} \lfloor raw/q \rfloor & raw \geq 0 \\ -\lfloor (-raw)/q \rfloor & raw < 0 \end{cases} \quad rem = raw - q \cdot t \in [0, q) \quad (4)$$

对 int64 的 floor 除法, rem 已落在 $[0, q)$, 数学上无需再做 $rem \geq q / rem < 0$ 的归一校正。

历史背景 (ntt_study / 交付 ONNX 写法): 早期实验环境与部分 Div 向量实现存在底层正确性问题, 交付图与 `mlkem_ref.stage31_mod` 在 rem 计算后仍保留

$$rem \leftarrow rem - q \cdot \mathbf{1}_{rem \geq q}, \quad rem \leftarrow rem + q \cdot \mathbf{1}_{rem < 0} \quad (5)$$

作为双校正兜底。本仓库基于 **AscendC 9.0.0** 工具链开发, **先默认 Div/标量整除无已知底层错误**; AscendC 标量路径仅用单次 $rem = raw - q \cdot t$ 。Golden 仍调用 `stage31_mod` (与交付 ONNX 字符串一致); 在正确整除前提下, 额外两步对合法输入为恒等, 与内核结果一致。

2 向量实现规划

2.1 AI Core 规模与验证档位

项	本用例约定
核类型	KERNEL_TYPE_AIV_ONLY: 仅使用 Vector AI Core (AIV) , 不使用 Cube AI Core (AIC) 。
计划 AI Core 数	三档: $aiv=1/2/8 \Rightarrow \text{blockDim}=1/2/8$ (<code>launch_profile.h + run.sh --aiv</code>)。
命名约定	LAUNCH_PROFILE=aiv=N; <code>run.sh --aiv 1 2 8</code> (与 Stage1 一致)。

数据划分 (aiv=8)	每核 1 poly: blockIdx= p ; 读 mat_c 行 $p, 8 + p$; 写 out 行 p 。
数据划分 (aiv=2)	每核 4 poly: blockIdx=0,1, 核内 $p = 4 \cdot blockIdx.. + 3$ 串行; 每 poly 新建 op。
数据划分 (aiv=1)	单核串行 8 poly; 每 poly 新建 op。
每核工作量	每条 poly 256 系数; kTileLen=16、双缓冲流水 (见下节)。
融合算子 AIV 约束	含 Cube 融合时 AIV 数须为 2 的倍数 ; 本用例增测 aiv=2 (每核 4 poly) 对齐最小偶数多核场景。
多档验证	默认三档均测 : aiv=1、aiv=2、aiv=8; 亦可 --aiv 单档。
验证命令	gen_data.py → 各档 ascendc_kernels_bbit → md5sum/cmp/verify_result.py。

选型理由:

- **aiv=8**: 满核 poly 并行; **aiv=2**: 偶数 AIV、每核 4 poly, 对齐融合 MIX 约束; **aiv=1**: 单核基线。
- 算术含标量 int64 (RouteA + mod), 多档主要验证 DataCopy 分核与 GM 偏移正确性。
- 核入口与 Stage1 同构: blockNum==1 串行 8 poly; 否则按 8/blockNum 均分 poly。

2.2 分块与 Gather 策略

- kTileLen=16、kPairCols=32: 每 tile 覆盖输出列 $j_0..j_0 + 31$ (每列对应一对偶/奇 mat_c 列)。
- 从 GM 连续拷贝 int32 对列块: HI 行偏移 $p \cdot 512 + 2j_0$, 长度 $2 \times kTileLen$ (含 hh, lh 交错); LO 行同理得 hl, ll 交错。
- **解交错**: 在 UB 小缓冲上按索引 $2i, 2i + 1$ 拆出四路 int32——**标量** (32 次/ tile)。
- 备选 (未在本版实现): DataCopyPad + DataCopyExtParams 跨步搬运 (07_0265), 或 Gather 类 ISASI API。

3 全量 API 清单

说明: 长表须用 L{} 列 + \apiname 断行; **禁止**多 API 挤在同一单元格 (见 Stage1 customspect)。

API	分类	在线文档 ID	A2	输入/输出 dtype (本用例)	备注
GlobalTensor	2.2 数据结构	07_0007	✓	GM 张量	SetGlobalBuffer
LocalTensor	2.2 数据结构	07_0006	✓	UB 张量	AllocTensor/GetValue

API	分类	在线文档 ID	A2	输入/输出 dtype (本 用例)	备注
GetBlockIdx	2.3 系统 变量	07_0185	✓	核索引	poly 划分
KERNEL_TASK_ TYPE_DEFAULT	Utils / 核类型	见编程指南	✓	AIV_ONLY	纯向量核
TPipe	2.3 资源 管理	07_0108	✓	管道	每 poly 新建
InitBuffer	2.3 资源 管理	07_0110	✓	UB 分配	双缓冲队列
TQue	2.3 Pipe/Que	07_0137	✓	VECIN/VEOUT	HI/LO 输入、out 输出
DataCopy	2.3.1 搬 运	07_0101	✓	int32 GM↔UB	连续对列块
DataCopyPad	2.3.1 ISASI	07_0265	✓	int32	备选 跨步 gather
Add	2.3.3 矢 量	07_0035	✓	int32	$hl + lh$; 见覆盖率
Muls	2.3.3 矢 量	07_0055	✓	int32	$\times 4096, \times 64$; 见覆盖率
Mul	2.3.3 矢 量	07_0037	✓	int32	Muls 等价备选
Cast	2.3.3 类 型转换	07_0073	✓	int32→int64	后续 int64 算术缺
Div	2.3.3 矢 量	07_0038	✓	half/float	无 int32/int64
Compare	2.3.3 矢 量	07_0066	✓	float/half; int32 仅 EQ	双校正比较
CompareScalar	2.3.3 矢 量	07_0068	✓	同上	与 q 、0 比较
Select	高阶 API	07_0859	✓	掩码选路	需 float 链
Sub	2.3.3 矢 量	07_0036	✓	int32	float 链 $rem - q \cdot t$

4 逐步覆盖率评估

步	数学	首选 API	覆盖	标量/备选
G1	读 C 对列块	DataCopy	100%	—

G2	拆 hh, hl, lh, ll	DataCopyPad 跨步	0% (本版)	标量解交错
R1	$hl + lh$	Add	条件	int32 可向量; 见 R3
R2	$\times 4096, \times 64$	Muls	条件	乘积可超 int32
R3	raw int64 合并	Cast+ 矢量算术	0%	标量 int64
M1	$\lfloor raw/q \rfloor$ 等	Div	0%	标量; 仅 float/half
M2	rem 归一 (历史)	Compare+Select	0%	本工程标量单次 rem
O1	写 out	DataCopy	100%	—

4.1 为何 int32 向量不能安全承担 raw

单路 mat_c 元素可达 $\sim 2 \times 10^6$ 量级 (256 路 int8 点积)。则 $hh \cdot 4096$ 可超过 2^{31} , **int32 向量乘加会溢出**; Python 使用 int64。dav_c220 上 Muls/Add 不支持 int64_t。

4.2 Stage3.1 向量备选 (未采用)

1. **float 链**: $Cast(int32 \rightarrow float) \rightarrow Div \rightarrow Cast \rightarrow Compare/Select$ 。风险: 大整数除法/取整与 ONNX Div 双校正逐元素一致需范围证明。
2. **定点乘法**: 类似 `mlkem_reduce_to_zq` 的移位近似 (需单独验证与 `stage31_mod` 等价)。

4.3 本版实现闭环结论

- **向量部分**: DataCopy 流水、核间划分、输出写回 ($\approx$ 搬运阶段 100%)。
- **标量部分**: 每系数 int64 RouteA 合并 + Stage31ModI64 (与融合内核 `sepolyvec8_ntt_custom.cpp` 同语义)。
- **总体向量覆盖率**: 算术主路径约 30%–40% (按指令条数粗估); **功能正确性 100%** 依赖标量缺口。

5 实现与验证

- 内核: `f203_stage3_routea_mod_custom.cpp` (AIV_ONLY, blockDim=8, 见 §2.1)
- Host: `main.cpp` (kBlockDim=8; ICPU_RUN_KF/f203_stage3_routea_mod_do)
- Golden: `f203_stage3_route_a + stage31_mod` (`scripts/gen_data.py`)
- 验证: `bash run.sh -r cpu -v Ascend910B4` (默认 aiv=1/2/8 三档)